

# GUÍA DE LABORATORIO 2

## Pruebas Unitarias del Servicio – notes-api

Proyecto: notes-api-db

Materia: Verificación y Validación de Software (INF780)

Nivel: Universitario | Duración: 3 horas

Herramientas: NestJS · Jest · TypeScript

2026

## 1. Objetivos

Al finalizar esta guía el estudiante será capaz de:

- Configurar el módulo de pruebas de NestJS para aislar el servicio mediante un repositorio mock.
- Aplicar la función `jest.fn()` y `mockResolvedValue()` para controlar el comportamiento de las dependencias.
- Escribir casos de prueba unitaria para todos los métodos del servicio: `create`, `findAll`, `findOne`, `update` y `remove`.
- Verificar valores de retorno, conteo de invocaciones y lanzamiento de excepciones con los matchers de Jest.
- Ejecutar las pruebas y analizar el reporte de cobertura generado.

## 2. Conceptos Clave

### 2.1 El patrón de aislamiento con mocks

Una prueba unitaria verifica una única unidad de código de forma aislada, sin dependencias externas reales. En este proyecto, el `NotesService` depende de `NotesRepository` para acceder a la base de datos. Si en la prueba se usara el repositorio real, la prueba dependería de PostgreSQL, sería lenta e impredecible.

La solución es reemplazar el repositorio real por un objeto simulado (mock) que devuelve valores controlados. NestJS facilita esto con `Test.createTestingModule()`, que permite registrar cualquier proveedor usando un valor alternativo mediante `useValue`.

**DEF**

Mock: objeto que imita la interfaz de una dependencia real pero devuelve valores prefijados. En Jest se construye con `jest.fn()`, una función falsa que registra sus llamadas, argumentos y puede configurarse para devolver cualquier valor.

### 2.2 Matchers utilizados en el archivo de pruebas

| Matcher                                    | Qué verifica                                                     |
|--------------------------------------------|------------------------------------------------------------------|
| <code>toEqual(valor)</code>                | Igualdad profunda de objetos y arrays (compara campo a campo).   |
| <code>toHaveBeenCalledWith(...args)</code> | Que el mock fue invocado con exactamente esos argumentos.        |
| <code>toHaveBeenCalledTimes(n)</code>      | Número exacto de veces que el mock fue llamado.                  |
| <code>toHaveLength(n)</code>               | Longitud de un array o string.                                   |
| <code>toBe(valor)</code>                   | Igualdad estricta por referencia o valor primitivo.              |
| <code>rejects.toThrow(Clase)</code>        | Que una promesa rechazada lanza una excepción del tipo indicado. |
| <code>rejects.toThrow("mensaje")</code>    | Que el mensaje de la excepción lanzada es exactamente ese texto. |

`not.toHaveBeenCalled()`

Que el mock nunca fue invocado durante la prueba.

## 2.3 Ciclo de vida de las pruebas

El archivo usa cuatro bloques de Jest para organizar el ciclo de vida:

| Bloque                    | Cuándo se ejecuta                 | Propósito en esta guía                                                                            |
|---------------------------|-----------------------------------|---------------------------------------------------------------------------------------------------|
| <code>describe()</code>   | Agrupar casos relacionados        | Un bloque por método del servicio ( <code>#create</code> , <code>#findAll</code> , etc.)          |
| <code>beforeEach()</code> | Antes de cada <code>it()</code>   | Recrea el <code>TestingModule</code> y el mock limpio para cada prueba                            |
| <code>afterEach()</code>  | Después de cada <code>it()</code> | Limpia el estado de todos los mocks con <code>jest.clearAllMocks()</code>                         |
| <code>it()</code>         | Cada caso individual              | Define un escenario concreto con su <code>arrange</code> , <code>act</code> y <code>assert</code> |

**NOTA**

Recrear el módulo en `beforeEach()` (no solo limpiar) garantiza que cada prueba parte de un estado completamente nuevo, evitando interferencias entre casos.

## 3. Desarrollo

El repositorio del proyecto está disponible en:

<https://github.com/HuascarFedor/INF780-NotesApiDb>

Clone el repositorio, instale las dependencias y cree el archivo de pruebas en la ruta indicada:

```
git clone https://github.com/HuascarFedor/INF780-NotesApiDb.git
cd INF780-NotesApiDb
npm install

# El archivo de pruebas va en:
src/notes/notes.service.spec.ts
```

**1**

### Importaciones y datos de prueba

Abra el archivo `src/notes/notes.service.spec.ts` (créelo si no existe) y agregue las importaciones y el objeto `mockNote` que servirá como fixture para todos los casos.

```
import { NotFoundException } from '@nestjs/common';
import { Test, TestingModule } from '@nestjs/testing';
import { NotesService } from '../notes.service';
import { NotesRepository } from '../notes.repository';
```

```
import { Note } from './entities/note.entity';
import { CreateNoteDto } from './dto/create-note.dto';
import { UpdateNoteDto } from './dto/update-note.dto';

const mockNote: Note = {
  id: '123e4567-e89b-12d3-a456-426614174000',
  title: 'Test Note',
  content: 'Test content',
  isArchived: false,
  createdAt: new Date('2024-01-01T00:00:00.000Z'),
  updatedAt: new Date('2024-01-01T00:00:00.000Z'),
};
```

**NOTA**

mockNote es el fixture de la prueba: un objeto Note con datos fijos y conocidos. Al usarlo como valor de retorno del mock, las aserciones toEqual(mockNote) siempre comparan contra un valor predecible.

**2****Tipo MockRepository y función factory**

Defina el tipo MockRepository y la función createMockRepository(). Estos dos elementos garantizan que el objeto mock tenga exactamente los mismos métodos que el repositorio real.

```
type MockRepository = {
  [K in keyof NotesRepository]: jest.Mock;
};

const createMockRepository = (): MockRepository => ({
  create: jest.fn(),
  findAll: jest.fn(),
  findOne: jest.fn(),
  update: jest.fn(),
  remove: jest.fn(),
});
```

**DEF**

El tipo [K in keyof NotesRepository]: jest.Mock; es un mapped type de TypeScript. Recorre cada clave del repositorio real y la reemplaza por jest.Mock. Si el repositorio cambia (se agrega o renombra un método), TypeScript generará un error de compilación en el mock, manteniendo ambos sincronizados automáticamente.

**3****Configuración del módulo de pruebas**

Defina el bloque describe principal con las variables compartidas, beforeEach() para crear el módulo y afterEach() para limpiar los mocks.

```
describe('NotesService', () => {
  let service: NotesService;
  let repository: MockRepository;
```

```
beforeEach(async () => {
  repository = createMockRepository();

  const module: TestingModule = await Test.createTestingModule({
    providers: [
      NotesService,
      { provide: NotesRepository, useValue: repository },
    ],
  }).compile();

  service = module.get<NotesService>(NotesService);
});

afterEach(() => {
  jest.clearAllMocks();
});

// ... bloques describe por método
});
```

La clave está en la línea `{ provide: NotesRepository, useValue: repository }`: le indica al módulo de NestJS que cuando algún proveedor solicite `NotesRepository` como dependencia, debe entregar el objeto mock en lugar de instanciar el repositorio real.

**4****Pruebas de #create — 1 caso**

El método `create()` del servicio simplemente delega al repositorio. La prueba verifica que: (a) el repositorio fue llamado con el DTO correcto, (b) fue llamado exactamente una vez, y (c) el valor devuelto es `mockNote`.

```
describe('#create', () => {
  it('should save and return the created note', async () => {
    const dto: CreateNoteDto = { title: 'Test Note', content: 'Test content' };
    repository.create.mockResolvedValue(mockNote);

    const result = await service.create(dto);

    expect(repository.create).toHaveBeenCalledWith(dto);
    expect(repository.create).toHaveBeenCalledTimes(1);
    expect(result).toEqual(mockNote);
  });
});
```

**NOTA**

`mockResolvedValue(mockNote)` configura la función mock para que, cuando sea llamada, devuelva una promesa resuelta con `mockNote`. Esto simula el comportamiento asíncrono del repositorio real sin tocar la base de datos.

**5****Pruebas de #findAll — 3 casos**

findAll() acepta un parámetro opcional isArchived. Se necesitan tres casos: sin filtro (undefined), con archived=true y con archived=false. En cada uno se verifica que el repositorio recibió exactamente el argumento que pasó el servicio.

```
describe('#findAll', () => {

  it('should return all notes when no filter is provided', async () => {
    repository.findAll.mockResolvedValue([mockNote]);
    const result = await service.findAll();
    expect(repository.findAll).toHaveBeenCalledWith(undefined);
    expect(result).toEqual([mockNote]);
  });

  it('should filter archived notes when archived=true', async () => {
    const archivedNote: Note = { ...mockNote, isArchived: true };
    repository.findAll.mockResolvedValue([archivedNote]);
    const result = await service.findAll(true);
    expect(repository.findAll).toHaveBeenCalledWith(true);
    expect(result).toHaveLength(1);
    expect(result[0].isArchived).toBe(true);
  });

  it('should filter non-archived notes when archived=false', async () => {
    repository.findAll.mockResolvedValue([mockNote]);
    const result = await service.findAll(false);
    expect(repository.findAll).toHaveBeenCalledWith(false);
    expect(result).toEqual([mockNote]);
  });

});
```

El spread operator { ...mockNote, isArchived: true } crea una copia superficial de mockNote sobrescribiendo solo isArchived. Esto evita modificar el fixture original que otros casos de prueba necesitan intacto.

**6****Pruebas de #findOne — 2 casos**

findOne() es el primer método con lógica de negocio: lanza NotFoundException si el repositorio devuelve null. Se prueban los dos caminos: nota encontrada y nota no encontrada.

```
describe('#findOne', () => {

  it('should return the note when it exists', async () => {
    repository.findOne.mockResolvedValue(mockNote);
    const result = await service.findOne(mockNote.id);
    expect(repository.findOne).toHaveBeenCalledWith(mockNote.id);
    expect(result).toEqual(mockNote);
  });

});
```

```
});

it('should throw NotFoundException when the note does not exist', async () => {
  repository.findOne.mockResolvedValue(null);

  await expect(service.findOne('non-existent-uuid'))
    .rejects.toThrow(NotFoundException);

  await expect(service.findOne('non-existent-uuid'))
    .rejects.toThrow('Note with id "non-existent-uuid" not found');
});

});
```

El caso de error invoca `service.findOne()` dos veces con dos aserciones independientes: la primera verifica el tipo de la excepción (`NotFoundException`) y la segunda verifica el mensaje exacto. Ambas aserciones deben preceder de `await` porque la excepción viene de una promesa rechazada.

**7****Pruebas de #update — 2 casos**

`update()` primero llama a `findOne()` internamente para verificar que la nota existe, y solo si existe llama a `repository.update()`. El caso de error debe verificar además que `repository.update` no fue llamado.

```
describe('#update', () => {

  it('should return the updated note when it exists', async () => {
    const dto: UpdateNoteDto = { title: 'Updated Title' };
    const updatedNote: Note = { ...mockNote, title: 'Updated Title' };
    repository.findOne.mockResolvedValue(mockNote);
    repository.update.mockResolvedValue(updatedNote);

    const result = await service.update(mockNote.id, dto);

    expect(repository.update).toHaveBeenCalledWith(mockNote.id, dto);
    expect(result).toEqual(updatedNote);
  });

  it('should throw NotFoundException when the note does not exist', async () => {
    repository.findOne.mockResolvedValue(null);

    await expect(service.update('non-existent-uuid', { title: 'x' }))
      .rejects.toThrow(NotFoundException);

    expect(repository.update).not.toHaveBeenCalled();
  });

});
```

**NOTA**

En el happy path se configura `findOne` para devolver `mockNote` (esto satisface la llamada interna a `this.findOne()` que hace el servicio) y luego se configura `update` para devolver la nota actualizada. Sin el mock de `findOne`, la llamada interna lanzaría `NotFoundException` antes de llegar al repositorio.

**8****Pruebas de #remove — 2 casos**

`remove()` sigue el mismo patrón que `update()`: verifica la existencia antes de eliminar. La diferencia es que no devuelve valor (`void`), por lo que el happy path verifica solo que el repositorio fue llamado exactamente una vez con el id correcto.

```
describe('#remove', () => {

  it('should call remove exactly once with the correct id', async () => {
    repository.findOne.mockResolvedValue(mockNote);
    repository.remove.mockResolvedValue(undefined);

    await service.remove(mockNote.id);

    expect(repository.remove).toHaveBeenCalledTimes(1);
    expect(repository.remove).toHaveBeenCalledWith(mockNote.id);
  });

  it('should throw NotFoundException when the note does not exist', async () => {
    repository.findOne.mockResolvedValue(null);

    await expect(service.remove('non-existent-uuid'))
      .rejects.toThrow(NotFoundException);

    expect(repository.remove).not.toHaveBeenCalled();
  });
});
```

**9****Ejecutar las pruebas y verificar cobertura**

Desde la raíz del proyecto ejecute los siguientes comandos. Todos los casos deben pasar en verde.

```
# Ejecutar solo las pruebas del servicio
npm run test -- notes.service

# Modo watch (re-ejecuta al guardar el archivo)
npm run test:watch -- notes.service

# Con reporte de cobertura
npm run test:cov
```



La salida esperada al ejecutar las pruebas del servicio es:

```
PASS   src/notes/notes.service.spec.ts
NotesService
  #create
    ✓ should save and return the created note
  #findAll
    ✓ should return an array of all notes when no filter is provided
    ✓ should filter archived notes when archived=true
    ✓ should filter non-archived notes when archived=false
  #findOne
    ✓ should return the note when it exists
    ✓ should throw NotFoundException when the note does not exist
  #update
    ✓ should return the updated note when it exists
    ✓ should throw NotFoundException when the note does not exist
  #remove
    ✓ should call remove exactly once with the correct id
    ✓ should throw NotFoundException when the note does not exist

Tests: 10 passed, 10 total
```

El reporte de cobertura del servicio debe mostrar 100% en todas las columnas:

```
-----|-----|-----|-----|-----|
File           | % Stmts | % Branch | % Funcs | % Lines |
-----|-----|-----|-----|-----|
notes/notes.service.ts |    100  |    100   |    100   |    100   |
-----|-----|-----|-----|-----|
```

## 4. Rúbrica de Evaluación

| Criterio                   | Indicadores                                                                                                | Pts | Obtenido |
|----------------------------|------------------------------------------------------------------------------------------------------------|-----|----------|
| Fixture y tipos de soporte | mockNote con los 6 campos;<br>MockRepository con mapped<br>type; createMockRepository()<br>con 5 jest.fn() | 15  |          |
| Configuración del módulo   | beforeEach recrea el módulo<br>con useValue; afterEach limpia<br>con clearAllMocks                         | 15  |          |
| Pruebas de #create         | Verifica<br>toHaveBeenCalledWith,<br>toHaveBeenCalledTimes(1) y<br>valor de retorno                        | 10  |          |

|                                      |                                                                                     |            |  |
|--------------------------------------|-------------------------------------------------------------------------------------|------------|--|
| <b>Pruebas de #findAll (3 casos)</b> | Cubre undefined, true y false; verifica argumento pasado al repositorio             | <b>15</b>  |  |
| <b>Pruebas de #findOne (2 casos)</b> | Happy path y NotFoundException con tipo y mensaje exacto                            | <b>15</b>  |  |
| <b>Pruebas de #update (2 casos)</b>  | Happy path con mock de findOne; error verifica not.toHaveBeenCalled()               | <b>15</b>  |  |
| <b>Pruebas de #remove (2 casos)</b>  | Happy path verifica toHaveBeenCalledTimes(1); error verifica not.toHaveBeenCalled() | <b>10</b>  |  |
| <b>Cobertura 100%</b>                | npm run test:cov reporta 100% en todas las columnas de notes.service.ts             | <b>5</b>   |  |
| <b>TOTAL</b>                         |                                                                                     | <b>100</b> |  |

**IMPORTANTE**

Estas pruebas unitarias verifican la lógica de negocio del servicio de forma aislada. No prueban la integración con PostgreSQL ni la capa HTTP. Esas capas se cubrirán en las guías de pruebas de integración y e2e.